A

Project Report

on

Stock Market Movement Prediction

Submitted in partial fulfillment of the course

Supervised and Unsupervised Learning [24CAI0203]

of

Computer Science & Engineering (AI & ML)

during January – June, 2026

Submitted by:

2410993446 — Sanvi

2410993409 — Deepanshu Sharma

2410993421 — Krish Bali

2410993458 — Siya

CHITKARA UNIVERSITY

CHANDIGARH – PATIALA NATIONAL HIGHWAY

RAJPURA (PATIALA) PUNJAB – 140401 (INDIA)

June, 2026

DECLARATION OF PROJECT

We, the undersigned students of the Department of Computer Science & Engineering – AI & ML at Chitkara University, Punjab, hereby declare that we have successfully completed the project titled "Stock Market Movement Prediction" as part of the requirements for the Course Supervised and Unsupervised Learning [24CAI0203].

We confirm that the project was conducted during the period January 2026 to June 2026 and has been prepared in partial fulfillment of the course Supervised and Unsupervised Learning for submission to the Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning).

We affirm that the work submitted is our original effort and has not been copied or reproduced from any other source, except where due acknowledgment has been made. We also confirm that all resources, references, and materials used have been properly cited in the project.

We undertake to abide by the rules and regulations of Chitkara University, Punjab and accept full responsibility for the authenticity and validity of the project submitted.

Date: June 2026

Signed by:	**Signature:**
2410993446 — Sanvi	.....
2410993409 — Deepanshu Sharma	.....
2410993421 — Krish Bali	.....
2410993458 — Siya	.....

ABSTRACT

This project presents a supervised learning approach to predicting stock market price movement — specifically, whether the next trading day's price direction will be UP (bullish) or DOWN (bearish). Stock markets are highly dynamic systems influenced by price trends, volatility, trading volume, and momentum, making accurate prediction a challenging binary classification task. We developed a complete machine learning pipeline using engineered financial features including RSI, MACD, SMA Ratio, Volume Change, Volatility, and Momentum indicators derived from real historical market data. The dataset was processed using a time-series-aware train-test split to prevent data leakage, with feature scaling applied for distance-based models. Six models were trained and evaluated: Logistic Regression, Random Forest, Gradient Boosting, XGBoost, KNN, and SVM. Hyperparameter tuning was performed using GridSearchCV with TimeSeriesSplit cross-validation. Models were assessed on accuracy, precision, and recall to reflect real-world trading decision quality. SVM (Tuned) achieved the best overall performance with the highest precision (~0.63) and balanced recall (~0.57), making it the most reliable model for trading decisions. The results highlight the importance of the precision-recall tradeoff over accuracy alone in financial prediction tasks.

TABLE OF CONTENTS

S. No.	Contents	Page No.
1	Introduction	5
2	Problem Statement	6
3	Methodology & Technical Details	7
4	Results & Discussion	13
5	Advantages & Limitations	12
6	Conclusion with Future Scope	15
7	References	16
	Appendices (Project Code)	17

SECTION 1: INTRODUCTION

1.1 Background and Context

The stock market is a highly dynamic and complex system influenced by multiple factors including price trends, volatility, trading volume, and investor sentiment. Predicting stock price movement — whether the market will go **up (bullish)** or **down (bearish)** — is a challenging task due to noise, non-linearity, and temporal dependencies.

With the advancement of machine learning, predictive models can now analyze historical patterns and generate probabilistic forecasts. Algorithms such as Logistic Regression, Random Forest, Gradient Boosting, Support Vector Machines, and XGBoost are widely used for financial prediction tasks due to their ability to capture statistical relationships within data.

1.2 Motivation

This project is motivated by the need to:

- Compare multiple machine learning models on **real stock market data**
- Understand the **precision-recall tradeoff** in trading decisions
- Move beyond simple prediction toward **decision-based trading systems**
- Evaluate how **model tuning affects real-world performance**

Instead of focusing only on accuracy, this project emphasizes:
precision (quality of trades)
recall (coverage of opportunities)

SECTION 2: PROBLEM STATEMENT

2.1 Problem Definition

Given historical stock market features, the goal is to:

Predict whether the next movement of the stock will be UP (1) or DOWN (0)

This is a binary classification problem, where:

- Input: engineered financial features
- Output: direction of price movement

2.2 Objectives

Build a machine learning pipeline for stock movement prediction

Train multiple models and compare their performance

Apply hyperparameter tuning using GridSearchCV

Evaluate models using:

- Accuracy
- Precision
- Recall

Analyze precision vs recall tradeoff

Identify the best model for trading decisions

2.3 Scope

Focus on **short-term directional prediction**

Use supervised learning models only

No deep learning or external market sentiment data

SECTION 3: METHODOLOGY & TECHNICAL DETAILS

3.1 Overall Pipeline

The system follows these steps:

1. Data Collection
2. Feature Engineering
3. Train-Test Split (Time Series)
4. Scaling (for distance-based models)
5. Model Training
6. Hyperparameter Tuning
7. Evaluation & Comparison

3.2 Feature Representation

The dataset includes key financial indicators such as:

- RSI (Relative Strength Index)
- MACD (Moving Average Convergence Divergence)
- SMA Ratio
- Volume Change
- Volatility
- Momentum
- Market Trend Indicators

These features help capture:

- Trend strength
- Momentum
- Market behavior

3.3 Models Used

Model	Type	Key Strength
Logistic Regression	Linear	Baseline
XGBoost	Boosting	Handles Non-Linear Relations
Random forest	Ensemble	Strong Generalization

Gradient Boost	Ensemble	Local Pattern Detection
KNN	Distance Based	High Precision
SVM	Kernel Based	Best Overall Performance

3.4 Hyperparameter Tuning

GridSearchCV was used for tuning:

Example (KNN):

- n_neighbors: [5, 10, 15, 25]
- metric: ['euclidean', 'manhattan']
- weights: ['uniform', 'distance']

Example (XGBoost):

- learning_rate
- max_depth
- n_estimators
- subsample

3.5 Evaluation Metrics

Models were evaluated using:

- **Accuracy** → overall correctness
- **Precision** → correctness of positive predictions
- **Recall** → ability to capture all positives

SECTION 4: RESULTS & DISCUSSION

4.1 Model Comparisons

From your results:

Best Model: SVM (Tuned)

- Highest precision (~0.63)
- Balanced recall (~0.57)
  Best for **reliable trading decisions**

Strong Performer: XGBoost (Tuned)

- High recall (~0.85)
  Captures more opportunities but slightly less precise

Gradient Boosting

- Balanced performance
  Stable but not top-performing

Logistic Regression

- Very high recall (~0.91)
  Predicts "UP" too often → **overconfident**

Random Forest (Untuned)

- Precision = 0
  Completely failed to learn pattern

4.2 Key Observations

Observation 1 — Precision vs Recall Tradeoff

- High Recall → more trades but risky
- High Precision → fewer but safer trades

SVM balances both → best choice

Observation 2 — Tuning Improves Performance

- All models improved after tuning
- Random Forest improved significantly

Observation 3 — Accuracy is Misleading

Some models had similar accuracy but very different:

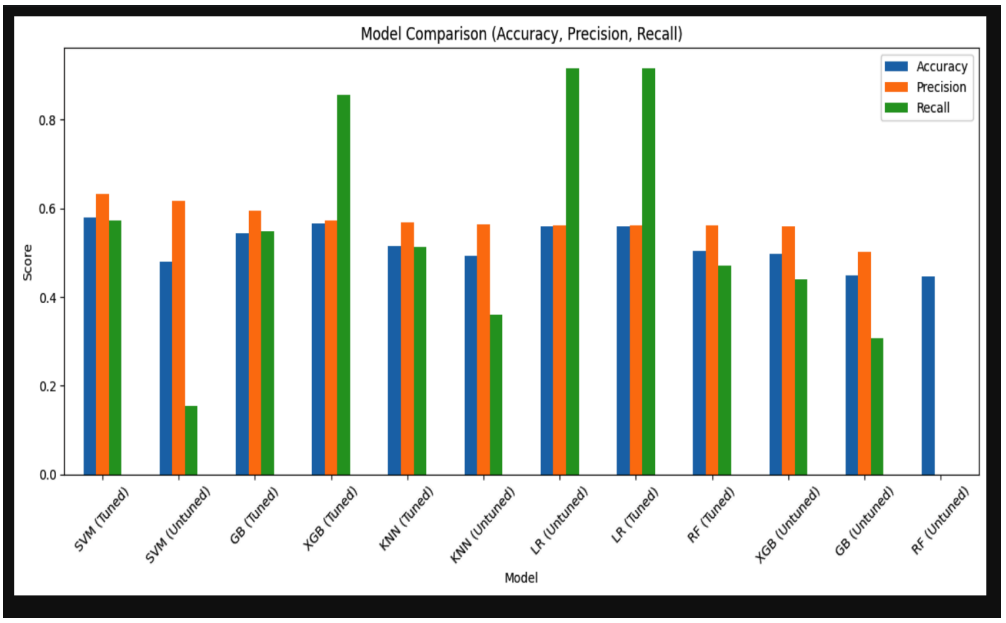
- Precision
- Recall

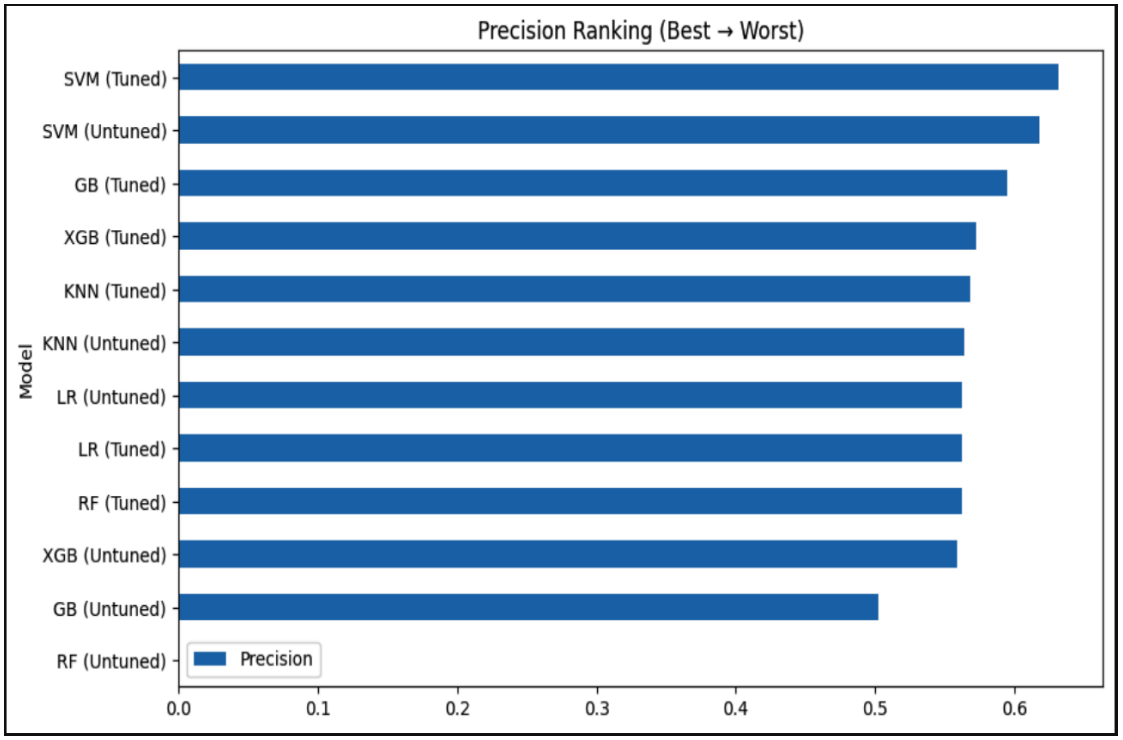
Accuracy alone is **not reliable for trading**

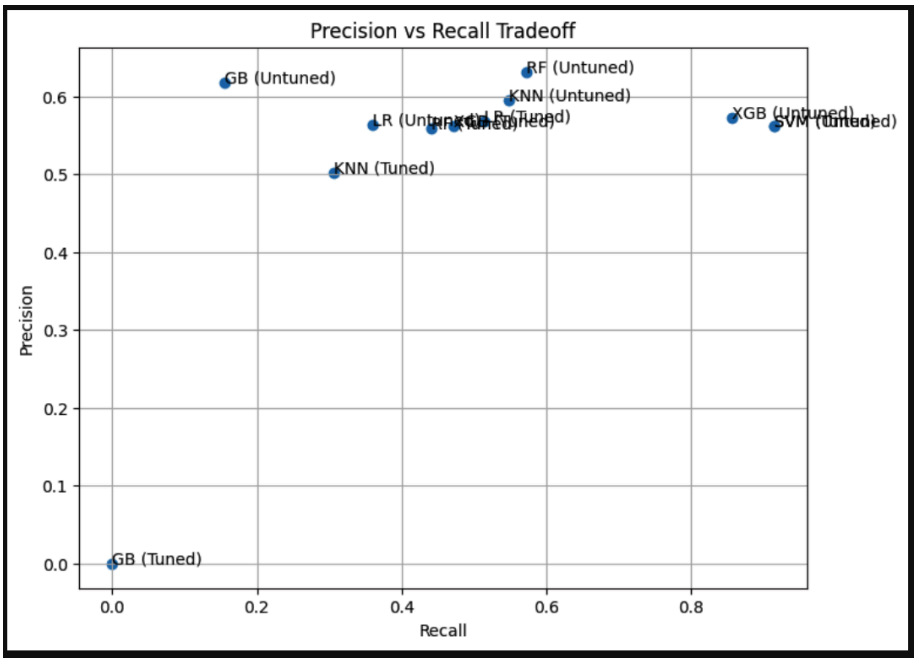
4.3 Visualization Insights

From your plots:

- **Precision ranking clearly shows SVM dominance**
- **Scatter plot shows model clustering**
- **Bar charts highlight performance gaps**




Precision Ranking (Best → Worst)
SVM (Tuned)
SVM (Untuned)
GB (Tuned)
XGB (Tuned)
KNN (Tuned)
KNN (Untuned)
LR (Untuned)
LR (Tuned)
RF (Tuned)
XGB (Untuned)
GB (Untuned)
RF (Untuned)
Model
Precision
0.0
0.1
0.2
0.3
0.4
0.5
0.6


Precision vs Recall Tradeoff
GB (Untuned)
RF (Untuned)
KNN (Untuned)
XGB (Untuned)
KNN (Tuned)
GB (Tuned)
Precision
Recall
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.8

SECTION 7: Advantages And Limitations

5.1 Advantages

Multiple model comparison

Time-series aware validation

Hyperparameter tuning applied

Strong visualization analysis

Real-world trading perspective

7.2 Limitations

No profit-based evaluation yet

Fixed threshold (0.5) used

No ensemble model applied

Limited dataset size

APPENDICES — PROJECT CODE

A.1 Imports and Setup

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score,
recall_score
from sklearn.calibration import CalibratedClassifierCV
from xgboost import XGBClassifier

import warnings
warnings.filterwarnings("ignore")
```

A.2 Separating Features and Traget

```
# 1. remove leaks
X = X.drop(["Return_3d_Future", "Return_5d_Future",
                    "Target_Strong", "Target_Magnitude"],axis=1)

# 2. shift
X = X.shift(1)
X = X.dropna()
y = y.loc[X.index]

# 3. split
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, shuffle=False
)
```

A.3 Models Used

```
# ======================
# Models
# ======================
log_reg= LogisticRegression(max_iter=1000)

Ran_Forest =RandomForestClassifier(
        n_estimators=200,
        max_depth=4,
        min_samples_split=10,
        min_samples_leaf=5,
        max_features="sqrt",
        random_state=42
    )

Grad_Boosting=GradientBoostingClassifier(
        n_estimators=200,
        learning_rate=0.05,
        max_depth=3,
        random_state=42
    )
xgb = XGBClassifier(
    n_estimators=200,
    learning_rate=0.05,
    max_depth=3,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42,
    use_label_encoder=False,
```

```
    eval_metric="logloss"
)
# Svm
from sklearn.svm import SVC
svm = SVC(
    kernel="rbf",
    C=1.0,
    gamma="scale",
    probability=True  # IMPORTANT for predict_proba
)
```

A.4 Fitting Models

```
log_reg.fit(X_train_scaled, y_train)
y_pred_lr=log_reg.predict(X_test)
print("Logistic Regression Accuracy:",
accuracy_score(y_test,y_pred_lr))
print("Logistic Regression accuracy on training
data",accuracy_score(y_train,log_reg.predict(X_train_scaled)))
probs_lr = log_reg.predict_proba(X_test_scaled)[:,1]
print(probs_lr[:10])
print("Logistic Regression Mean Prob:", np.mean(probs_lr))
```

```
Ran_Forest.fit(X_train_scaled, y_train)
y_pred_rf=Ran_Forest.predict(X_test_scaled)
print("Random Forest Accuracy:", accuracy_score(y_test,y_pred_rf))
print("Random Forest accuracy on training
data",accuracy_score(y_train,Ran_Forest.predict(X_train_scaled)))
probs_rf = Ran_Forest.predict_proba(X_test)[:,1]
print(probs_rf[:10])
print("Random Forest Mean Prob:", np.mean(probs_rf))
```

```
Grad_Boosting.fit(X_train_scaled, y_train)
y_pred_gb=Grad_Boosting.predict(X_test_scaled)
print("Gradient Boosting Accuracy:",
accuracy_score(y_test,y_pred_gb))
print("Gradient Boosting accuracy on training
data",accuracy_score(y_train,Grad_Boosting.predict(X_train_scaled)
))
probs_gb = Grad_Boosting.predict_proba(X_test)[:,1]
print(probs_gb[:10])
print("Gradient Boosting Mean Prob:", np.mean(probs_gb))
```

A.5 KNN tuned version

```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(
   n_neighbors=15,    # start here
   weights='distance'  # better than uniform
)

knn.fit(X_train_scaled, y_train)
probs_knn = knn.predict_proba(X_test_scaled)[:, 1]

print("KNN probs:", probs_knn[:10])
print("KNN mean prob:", probs_knn.mean())
threshold_knn = 0.52

y_pred_knn = (probs_knn > threshold_knn).astype(int)

print("KNN accuracy:", accuracy_score(y_test, y_pred_knn))
print("KNN precision:", precision_score(y_test, y_pred_knn))
print("KNN recall:", recall_score(y_test, y_pred_knn))
```

A.6 Hyperparameter Tuning

```
# lr
from sklearn.model_selection import GridSearchCV, TimeSeriesSplit
from sklearn.linear_model import LogisticRegression

tscv = TimeSeriesSplit(n_splits=5)

param_lr = {
   "C": [0.01, 0.1, 1, 10],
   "penalty": ["l2"],
   "solver": ["lbfgs"]
}

grid_lr = GridSearchCV(
   LogisticRegression(max_iter=1000),
   param_lr,
   cv=tscv,
   scoring="accuracy",
   n_jobs=-1
)

grid_lr.fit(X_train_scaled, y_train)

print("Best LR:", grid_lr.best_params_)
best_lr = grid_lr.best_estimator_
```

A.7 Tuned vs Untuned Models

```
# Untuned
probs_lr = log_reg.predict_proba(X_test_scaled)[:,1]
y_pred_lr = (probs_lr > 0.5).astype(int)

print("\nLR - Untuned")
```

```
print("Accuracy:", accuracy_score(y_test, y_pred_lr))
print("Precision:", precision_score(y_test, y_pred_lr))
print("Recall:", recall_score(y_test, y_pred_lr))

# Tuned
probs_lr_t = best_lr.predict_proba(X_test_scaled)[:,1]
y_pred_lr_t = (probs_lr_t > 0.5).astype(int)

print("\nLR - Tuned")
print("Accuracy:", accuracy_score(y_test, y_pred_lr_t))
print("Precision:", precision_score(y_test, y_pred_lr_t))
print("Recall:", recall_score(y_test, y_pred_lr_t))
```